

System Testing Report

1. Test Suite Overview

Unit Tests	Integration Tests	Total
38 tests-all pass	62 tests-all pass	100 tests-all pass

1.1 Unit Test Coverage

Module	Tests	Pass	Fail
Root	1	1	0
Auth	2	2	0
Auth Middleware	2	2	0
User	1	1	0
Admin Role Update	2	2	0
Admin Users	3	3	0
Notifications	3	3	0
Categories	3	3	0
Items	3	3	0
Availability	1	1	0
Booking	3	3	0
Condition Images	4	4	0
Request Status	1	1	0
Checkout & Return	2	2	0
Overdue	2	2	0
Password Reset	3	3	0
Admin Reports	2	2	0
TOTAL	38	38	0

1.2 Integration Test Coverage

Integration tests are run against the real local database (doesn't interfere with live database). Nodemailer is mocked to prevent emails being sent. All other dependencies are live.

Module	Tests	Pass	Fail
Auth-Register	3	3	0
Auth-Login	3	3	0
Auth-getUser	3	3	0
Items-Get	2	2	0
Items-CRUD	5	5	0
Items-Availability	3	3	0
Categories	5	5	0
Bookings-bookItem	2	2	0
Bookings-Lifecycle	11	11	0
Bookings-Queries	4	4	0
Notifications	5	5	0
Admin-Users	6	6	0
Admin-Faculties	5	5	0
Admin-Reports	5	5	0
TOTAL	62	62	0

2. Unit Test Cases

All unit tests mock the database and external services. They validate routing, middleware, and input validation logic in isolation.

ID	Module	Description	Steps	Expected	Status
TC-U01	Root	GET / returns API status	GET /	200 OK with message	Pass
TC-U02	Auth	Register-missing fields	POST empty body to /api/register	400 Bad Request	Pass
TC-U03	Auth	Login-missing fields	POST empty body to /api/login	400 Bad Request	Pass
TC-U04	Auth Middleware	Rejects request with no token	GET /api/getUser without token	401 Unauthorized	Pass
TC-U05	Auth Middleware	Rejects request with invalid token	GET /api/getUser with invalid token	403 Forbidden	Pass
TC-U06	User	Get user-authenticated request handled	GET /api/getUser with valid token	200 OK, user data returned	Pass
TC-U07	Admin Role Update	Rejects non-admin user	PUT /api/admin/users/1/role with Student token	403 Forbidden	Pass
TC-U08	Admin Role Update	Rejects invalid role value	PUT /api/admin/users/1/role with invalid role	400 Bad Request	Pass
TC-U09	Admin Users	Get all users-admin only	GET /api/admin/users with Admin token	200 OK, users listed	Pass
TC-U10	Admin Users	Get all users-rejects non-admin	GET /api/admin/users with Student token	403 Forbidden	Pass
TC-U11	Admin Users	Delete user-rejects non-admin	DELETE /api/admin/users/1 with Student token	403 Forbidden	Pass
TC-U12	Notifications	Get notifications	GET /api/notifications with valid token	200 OK, notifications returned	Pass
TC-U13	Notifications	Mark notification read-invalid ID	PUT /api/notifications/abc/read	400 Bad Request	Pass
TC-U14	Notifications	Mark all notifications read	PUT /api/notifications/read-all	200 OK	Pass
TC-U15	Categories	Get categories-authenticated	GET /api/categories with valid token	200 OK, categories returned	Pass
TC-U16	Categories	Create category-missing name	POST /api/categories with empty body	400 Bad Request	Pass
TC-U17	Categories	Create category-non-staff rejected	POST /api/categories as Student with valid name	403 Forbidden	Pass
TC-U18	Items	Get items	GET /api/items with Admin token	200 OK, items listed	Pass
TC-U19	Items	Create item-missing fields	POST /api/items with Faculty token, empty body	400 Bad Request	Pass
TC-U20	Items	Delete item-invalid ID	DELETE /api/items/abc with Admin token	400 Bad Request	Pass

TC-U21	Availability	Get availability-missing params	GET /api/items/availability without query params	400 Bad Request	Pass
TC-U22	Booking	Book item-missing fields	POST /api/book-item with empty body	400 Bad Request	Pass
TC-U23	Booking	Get my requests-authenticated	GET /api/my-requests with valid token	200 OK, requests returned	Pass
TC-U24	Booking	Cancel request-missing request_id	PUT /api/request-cancel with empty body	400 Bad Request	Pass
TC-U25	Condition Images	Get images by item-invalid ID	GET /api/condition-images/abc	400 Bad Request	Pass
TC-U26	Condition Images	Upload image by item-no file	POST /api/condition-images/1 without file	400 Bad Request	Pass
TC-U27	Condition Images	Get images by borrow request-invalid ID	GET /api/borrowrequest/abc/condition-images	400 Bad Request	Pass
TC-U28	Condition Images	Upload borrow request image-missing image_type	POST /api/borrowrequest/1/condition-image with empty body	400 Bad Request	Pass
TC-U29	Request Status	Update request status-missing fields	PUT /api/request-status with empty body	400 Bad Request	Pass
TC-U30	Checkout & Return	Checkout-missing request_id	PUT /api/request-checkout with empty body	400 Bad Request	Pass
TC-U31	Checkout & Return	Return-missing request_id	PUT /api/request-return with empty body	400 Bad Request	Pass
TC-U32	Overdue	Get overdue requests-admin	GET /api/overdue-requests with Admin token	200 OK, overdue requests listed	Pass
TC-U33	Overdue	Get overdue requests-unauthenticated	GET /api/overdue-requests without token	401 Unauthorized	Pass
TC-U34	Password Reset	Forgot password-missing email	POST /api/auth/forgot-password with empty body	400 Bad Request	Pass
TC-U35	Password Reset	Reset password-missing password	POST /api/auth/reset-password/token with empty body	400 Bad Request	Pass
TC-U36	Password Reset	Reset password-invalid token	POST /api/auth/reset-password/badtoken with password	400 Bad Request	Pass
TC-U37	Admin Reports	Reports summary-rejects non-admin	GET /api/admin/reports/summary with Student token	403 Forbidden	Pass
TC-U38	Admin Reports	Reports summary-admin access	GET /api/admin/reports/summary with Admin token	200 OK, report data returned	Pass

3. Integration Test Cases

Integration tests run against a real local MySQL database that mirrors the production schema. Each describe block seeds its own data and cleans up afterAll. Tests are run serially with `–runInBand` to avoid race conditions.

ID	Module	Description	Steps	Expected	Status
TC-I01	Auth-Register	Registers a new user and returns a JWT	POST /api/register with valid full payload	200 OK, token returned, user_type = Student	Pass
TC-I02	Auth-Register	Rejects registration with missing fields	POST /api/register with only email field	400 Bad Request	Pass
TC-I03	Auth-Register	Rejects duplicate email	POST /api/register using email already in DB	409 Conflict, error mentions email	Pass
TC-I04	Auth-Login	Logs in with correct credentials	POST /api/login with valid email and password	200 OK, token + full user object returned	Pass
TC-I05	Auth-Login	Rejects wrong password	POST /api/login with correct email, wrong password	401 Unauthorized	Pass
TC-I06	Auth-Login	Rejects non-existent email	POST /api/login with email not in DB	401 Unauthorized	Pass
TC-I07	Auth-getUser	Returns user profile for a valid token	GET /api/getUser with valid JWT	200 OK, user_id matches seeded user	Pass
TC-I08	Auth-getUser	Rejects missing token	GET /api/getUser with no Authorization header	401 Unauthorized	Pass
TC-I09	Auth-getUser	Rejects tampered token	GET /api/getUser with malformed JWT string	403 Forbidden	Pass
TC-I10	Items-Get	Returns items array for authenticated user	GET /api/items with valid JWT	200 OK, items is an array	Pass
TC-I11	Items-Get	Rejects unauthenticated request	GET /api/items with no token	401 Unauthorized	Pass
TC-I12	Items-CRUD	Faculty can add an item	POST /api/items with Faculty token, name + description fields	200 OK, message contains 'added'	Pass
TC-I13	Items-CRUD	Faculty can delete their own item	DELETE /api/items/:id with Faculty token, item owned by user	200 OK	Pass
TC-I14	Items-CRUD	Student cannot add an item	POST /api/items with Student token	403 Forbidden	Pass
TC-I15	Items-CRUD	Serial number longer than 15 chars is rejected	POST /api/items with Faculty token, serial_number = 23 chars	400 Bad Request	Pass
TC-I16	Items-CRUD	Faculty cannot edit an item they do not own	PUT /api/edit-item with Faculty token, item_id owned by another user	403 Forbidden	Pass

TC-I17	Items-Availability	Returns available items for a valid date range	GET /api/items/availability with future start and end dates	200 OK, items is an array	Pass
TC-I18	Items-Availability	Rejects when end is before start	GET /api/items/availability where end < start	400 Bad Request	Pass
TC-I19	Items-Availability	Rejects missing query params	GET /api/items/availability with no query string	400 Bad Request	Pass
TC-I20	Categories	Admin can create a category	POST /api/admin/categories with Admin token and name	200 OK, category inserted in DB	Pass
TC-I21	Categories	Authenticated user can list categories	GET /api/categories with Student token	200 OK, categories is an array	Pass
TC-I22	Categories	Admin can delete a category	DELETE /api/admin/categories/:id with Admin token	200 OK	Pass
TC-I23	Categories	Student cannot create a category	POST /api/categories with Student token	403 Forbidden	Pass
TC-I24	Categories	Duplicate category name returns 409	POST /api/admin/categories with name already in DB	409 Conflict	Pass
TC-I25	Bookings-bookItem	Booking with end before start returns 400	POST /api/book-item with end date earlier than start date	400 Bad Request	Pass
TC-I26	Bookings-bookItem	Booking a non-existent item returns 404	POST /api/book-item with item_id = 9999999	404 Not Found	Pass
TC-I27	Bookings-Lifecycle	Owner can approve a Pending request	PUT /api/request-status with Faculty token, status = Approved	200 OK, message contains 'updated'	Pass
TC-I28	Bookings-Lifecycle	Owner can reject a Pending request with a note	PUT /api/request-status with Faculty token, status = Rejected, decision_note set	200 OK	Pass
TC-I29	Bookings-Lifecycle	Borrower can cancel their own Pending request	PUT /api/request-cancel with borrower's Student token	200 OK	Pass
TC-I30	Bookings-Lifecycle	Owner can checkout an Approved request	PUT /api/request-checkout with Faculty token, request in Approved status	200 OK, message contains 'checked out'	Pass
TC-I31	Bookings-Lifecycle	Owner can mark a CheckedOut request as returned	PUT /api/request-return with Faculty token, request in CheckedOut status	200 OK, message contains 'returned'	Pass
TC-I32	Bookings-Lifecycle	Cannot approve an already-Approved request	PUT /api/request-status on Approved request	409 Conflict	Pass
TC-I33	Bookings-Lifecycle	Different user cannot cancel someone else's request	PUT /api/request-cancel with unrelated Student token	403 Forbidden	Pass
TC-I34	Bookings-Lifecycle	Cannot checkout a Pending request	PUT /api/request-checkout on Pending request	409 Conflict	Pass

TC-I35	Bookings-Lifecycle	Cannot return a Pending request	PUT /api/request-return on Pending request	409 Conflict	Pass
TC-I36	Bookings-Lifecycle	Decision note over 500 chars is rejected	PUT /api/request-status with decision_note of 501 characters	400 Bad Request	Pass
TC-I37	Bookings-Lifecycle	Non-owner faculty cannot approve request for another's item	PUT /api/request-status with Faculty token that does not own the item	403 Forbidden	Pass
TC-I38	Bookings-Queries	Borrower can fetch their own bookings	GET /api/my-requests with borrower's Student token	200 OK, seeded request appears in results	Pass
TC-I39	Bookings-Queries	Faculty can fetch item requests for their items	GET /api/item-requests with Faculty token	200 OK, requests is an array	Pass
TC-I40	Bookings-Queries	Admin can fetch all overdue requests	GET /api/overdue-requests with Admin token	200 OK, requests is an array	Pass
TC-I41	Bookings-Queries	Student cannot fetch item requests	GET /api/item-requests with Student token	403 Forbidden	Pass
TC-I42	Notifications	Returns notifications and unread count	GET /api/notifications with valid token, notification seeded in DB	200 OK, unreadCount >= 1	Pass
TC-I43	Notifications	Can mark a specific notification as read	PUT /api/notifications/:id/read, verify is_read = 1 in DB	200 OK, DB confirms is_read = 1	Pass
TC-I44	Notifications	Can mark all notifications as read	PUT /api/notifications/read-all with valid token	200 OK, ok = true	Pass
TC-I45	Notifications	Cannot fetch notifications without a token	GET /api/notifications with no Authorization header	401 Unauthorized	Pass
TC-I46	Notifications	Marking non-numeric notification ID returns 400	PUT /api/notifications/abc/read	400 Bad Request	Pass
TC-I47	Admin-Users	Admin can get all users	GET /api/admin/users with Admin token	200 OK, users is an array	Pass
TC-I48	Admin-Users	Admin can update a user's role	PUT /api/admin/users/:id/role, verify user_type in DB	200 OK, DB confirms updated role	Pass
TC-I49	Admin-Users	Admin can delete another user	DELETE /api/admin/users/:id with Admin token	200 OK	Pass
TC-I50	Admin-Users	Admin cannot delete their own account	DELETE /api/admin/users/:id where id = own user_id	400 Bad Request, error mentions own account	Pass
TC-I51	Admin-Users	Non-admin cannot access admin user routes	GET /api/admin/users with Student token	403 Forbidden	Pass
TC-I52	Admin-Users	Updating user with invalid role returns 400	PUT /api/admin/users/:id/role	400 Bad Request	Pass

			with user_type = Supervillain		
TC-I53	Admin-Faculties	Admin can list faculties	GET /api/admin/faculties with Admin token	200 OK, faculties is an array	Pass
TC-I54	Admin-Faculties	Admin can create a faculty	POST /api/admin/faculties with Admin token and name	200 OK, faculty inserted in DB	Pass
TC-I55	Admin-Faculties	Admin can delete a non-default faculty	DELETE /api/admin/faculties/:id with Admin token	200 OK	Pass
TC-I56	Admin-Faculties	Cannot delete the default General faculty (id=1)	DELETE /api/admin/faculties/1 with Admin token	400 Bad Request, error mentions default	Pass
TC-I57	Admin-Faculties	Duplicate faculty name returns 409	POST /api/admin/faculties with name already in DB	409 Conflict	Pass
TC-I58	Admin-Reports	Admin can fetch reports summary	GET /api/admin/reports/summary with Admin token	200 OK, statusCounts/topTools/topBorrowers arrays	Pass
TC-I59	Admin-Reports	Admin can get all requests	GET /api/admin/requests with Admin token	200 OK, requests is an array	Pass
TC-I60	Admin-Reports	Admin can filter requests by status	GET /api/admin/requests?status=Pending with Admin token	200 OK, all returned requests have status=Pending	Pass
TC-I61	Admin-Reports	Student cannot access admin reports	GET /api/admin/reports/summary with Student token	403 Forbidden	Pass
TC-I62	Admin-Reports	Student cannot access admin requests list	GET /api/admin/requests with Student token	403 Forbidden	Pass

4. Observations & Analysis

4.1 Error Handling Robustness

- All routes validate input correctly before processing.
- Missing fields, invalid IDs, and malformed requests are rejected with appropriate 400 status codes.
- Conflict scenarios (duplicate email, overlapping bookings, duplicate categories/faculties) correctly return 409.
- Attempting invalid status transitions (e.g. approving an already-approved request, checking out a pending one) returns 409 with a descriptive error.

4.2 Authorisation & Security

- JWT authentication middleware correctly rejects missing tokens (401) and invalid/tampered tokens (403) across all protected routes.
- Role-based access control is enforced for all admin-only and Faculty/Staff-only actions, Student tokens are blocked at the correct layer.
- Integration tests confirm that a Faculty member cannot edit or approve resources they do not own, verifying ownership checks at the database level.
- Admin self-deletion protection is confirmed via the integration suite by attempting to delete the authenticated admin's own account.

4.3 Endpoint Coverage

- Unit tests cover all major route groups: Auth, Users, Admin, Items, Categories, Booking, Notifications, Condition Images, Requests, Overdue, and Password Reset.
- Integration tests extend coverage to the full borrow request lifecycle: submission, approval, rejection, checkout, return, and cancellation against a live database.
- Integration tests verify database-level side effects, such as confirming a notification's `is_read` flag is actually set to 1 after a mark-read call.
- Admin filtering (e.g. `GET /api/admin/requests?status=Pending`) is verified to return only correctly-filtered results.

4.4 Test Isolation & Reliability

- Unit tests mock the database module, nodemailer, the overdue scheduler, and the password reset service to prevent side effects and ensure deterministic results.
- Integration tests mock only nodemailer to prevent real emails. All other interactions use the real database.
- Each integration block seeds its own data with unique timestamps and integer counters, and deletes it after everything is done, ensuring tests do not pollute each other.
- The `getAnyFacultyId()` helper dynamically resolves a valid `faculty_id` at runtime rather than hardcoding a value, making the suite portable across different database states.

5. Conclusion

All 100 tests across both suites passed successfully. The unit test suite confirms that input validation, authentication middleware, and role-based access control are functioning correctly in isolation. The integration test suite confirms that these behaviours hold against a real database, and additionally verifies the full borrow request lifecycle, database-level side effects, and ownership enforcement.

- The backend correctly validates input, enforces authentication, and applies user role restrictions throughout.
- The borrow request lifecycle (Pending -> Approved -> CheckedOut -> Returned) is handled robustly, with appropriate guards against invalid transitions.
- External services are mocked where appropriate, keeping the test suite reliable and suitable for continuous integration.
- The MVC architecture of the backend allows each layer to be tested independently, supporting both the mocked unit tests and the live integration tests without code changes.